

Adaptive Advance-Time Control for nFAPI Split under Unpredictable Latency

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1[†]

*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan

[†]Affiliation 2

Email: crg@mail.ntust.edu.tw

Abstract—

Index Terms—nFAPI, O-RAN, Functional Split, Adaptive Control.

I. Introduction

A. Background and Motivation

With the evolution of 5G and the Open Radio Access Network (O-RAN), architectures based on functional splits have become the key to improving network flexibility and reducing deployment costs. Currently, the most mainstream implementation promoted by the O-RAN Alliance is Split Option 7.2 (Split 7.2). Because Split 7.2 divides the Physical (PHY) layer into high and low parts and transmits continuous in-phase and quadrature (IQ) samples, it imposes extremely high requirements on the fronthaul network.

Compared to Split 7.2, the 3rd Generation Partnership Project (3GPP) Split Option 6 (Split 6), which separates the network at the Media Access Control (MAC) and PHY layers, offers a more flexible alternative. First, Split 7.2 relies on hardware-based synchronization protocols, such as IEEE 1588 Precision Time Protocol (PTP) and Synchronous Ethernet (SyncE), to achieve nanosecond-level phase and time synchronization. Its timing window requirement is typically strict, often needing to be less than 100 μ s. In contrast, the nFAPI (Network Functional Application Platform Interface) protocol, which is a prominent implementation of Split 6, employs a message-based synchronization mechanism. Through its proprietary P7 Node Sync procedure, nFAPI achieves slot-level synchronization (millisecond-level) and provides a highly flexible timing window ranging from 0 to 30,000 μ s. Furthermore, in terms of payload, Split 6 transports packetized MAC Protocol Data Units (PDUs) rather than continuous IQ samples. These characteristics allow Split 6 to demonstrate better adaptability in non-ideal fronthaul network environments where high jitter from operating system (OS) scheduling and network delivery is present.

This paper chooses Split 6 as the primary research subject due to its significant deployment benefits. As indicated by Pérez-Romero et al. [1], Split 6 effectively reduces the traffic burden on the fronthaul network. Compared to low-layer split schemes that transmit frequency-domain signals, its bandwidth requirement is significantly lower.

In addition, Khan et al. [2] demonstrate that for the deployment of sub-6 GHz small cell networks, Split 6 can maintain robust transmission performance over non-ideal fronthaul connections, thereby achieving an optimal cost-revenue trade-off.

Deploying the VNF and PNF on different machines introduces significant challenges. Because the MAC scheduler is separated from the PHY layer, strict slot synchronization is required between these distributed components. However, different traffic loads and non-ideal network deployments introduce unpredictable, non-deterministic latency. Consequently, the VNF cannot reliably transmit data exactly at the slot boundary, and existing open-source solutions suffer from synchronization failures when encountering unpredictable latency from routers and switches. This paper defines these latency and synchronization problems under non-ideal fronthaul networks and proposes a dynamic trigger scheduler adjustment algorithm. A detailed analysis of this non-deterministic latency and the corresponding verification of these problems will be presented in Section IV, specifically isolating these issues in Scenario I (IV-B).

B. Related Works

The evolution of Radio Access Network (RAN) architectures from Distributed RAN (D-RAN) to Open RAN (O-RAN) requires a thorough study of functional splits [3]–[5]. The MAC-PHY split (Option 6), standardized by the Small Cell Forum (SCF) as nFAPI [6], [7], is suitable for small cell deployments. It allows the MAC layer to be virtualized on commercial servers while the PHY layer remains on dedicated hardware.

The performance of the nFAPI split depends on the transport network. Practical deployments often use non-ideal fronthaul, such as Shared Ethernet or DOCSIS. Mehran and MacKenzie [8] demonstrated that uplink throughput drops to zero with a small increase in fronthaul delay. Furthermore, server processing delay is unpredictable. Khalaf et al. [9] indicated that server delay increases under high CPU utilization, causing the MAC scheduler to miss its timing window.

To address fronthaul latency, Huang and Zhang [10] proposed a distributed MAC scheduling scheme. While this scheme reduces the impact of delayed grants, it

TABLE I
Throughput Performance Comparison

TX Load	Baseline Configurations (Index 1–10)										Proposed
	1	2	3	4	5	6	7	8	9	10	
100 Mbps	–	100	100	100	100	100	100	100	100	100	100
1000 Mbps	–	–	761	882	891	901	923	908	885	846	932

Note: '–' indicates UE disconnection.

introduces a “split-brain” issue with multiple schedulers. Akgun et al. [11] utilized intelligent scheduling to adapt modulation based on interference, but did not solve the sub-millisecond timing of message delivery itself. Existing OpenAirInterface (OAI) reference models typically rely on static timing windows, leading to frequent “missing subframe” errors when network jitter exceeds configured limits. This paper bridges this gap by proposing a centralized Adaptive Advance-Time Control algorithm to dynamically manage scheduling timing.

C. Contributions

- Delay Management in nFAPI Split 6: We analyze and quantify the non-deterministic latency factors, including network jitter and server processing time, that lead to synchronization failures in disaggregated RAN deployments.
- Adaptive Advance-Time Control Algorithm: We propose a dynamic timing adjustment algorithm using a dual-loop architecture. The Convergence Optimization Loop estimates end-to-end jitter, while the VNF Timing Actuator adjusts scheduling lead times to ensure punctual delivery of P7 messages.
- Experimental Validation: We prototype the proposed mechanism on the OAI platform. Experimental results show that our algorithm eliminates “missing subframe” errors and maintains stable throughput under non-ideal fronthaul environments.

The rest of the paper is organized as follows. Section II defines the system model/architecture considered in this paper. The proposed analytical model/method is given in Section III. Section IV shows the numerical results. Concluding remarks are given in Section VI.

II. System Model/Architecture

Before diving into the system details, this study establishes a fundamental assumption: currently, there is a lack of Open Radio Access Network (O-RAN) Radio Units (O-RUs) in the market that natively support the Network Functional Application Platform Interface (nFAPI) via Split 6. As a result, the proposed system adopts a resilient two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to the remote O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- Core Network (CN): The central element responsible for routing, session management, and authenticating user data.
- O-RAN Central Unit (O-CU): A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- O-RAN Distributed Unit (O-DU): To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- Switch: A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- O-RAN Radio Unit (O-RU): A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- Commercial Off-The-Shelf User Equipment (COTS UE): Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC and Delay Manager

Within the Media Access Control (MAC) layer of the O-DU High, the core components handling timing mechanisms include the Scheduler and the nFAPI Delay Manager:

- Scheduler: Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- nFAPI Delay Manager: This resides as the crucial control unit of this delay-aware architecture. Because unpredictable, unstable latency exists in the switch between the VNF and PNF, the Delay Manager

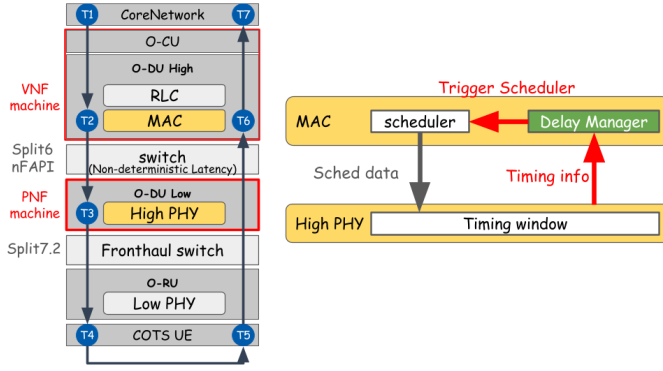


Fig. 1. System architecture and delay management mechanism.

continuously receives timing feedback from the lower physical layer. It uses this to dynamically calculate network jitter and proactively trigger the scheduler. This calculation guarantees that the scheduling instructions (Sched data) are generated ahead of time to account for network delays.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. When the scheduling message (specifically the `DL_TTI.request`) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the subframe data.
- **Timing Info:** The PNF continuously monitors the exact arrival timestamp of these control messages and periodically sends this “Timing Info” back to the Delay Manager in the MAC layer. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following three Key Validation Metrics, where RTT denotes the Round-Trip Time:

- **Ping RTT ($T_7 - T_1$):** Measures the end-to-end round-trip latency at the application layer. This metric captures the complete duration from when a packet is sent from the CN to when its return acknowledgment is received by the sender.
- **HARQ RTT ($T_6 - T_2$):** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically

at the MAC layer. HARQ RTT accurately reflects the feedback efficiency and operational health of the link layer.

- **VNF-PNF Downlink Latency ($T_3 - T_2$):** Measures the one-way downlink latency accumulated from the virtualized machine to the physical machine. This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

The following assumptions were made in this paper.

- Assumption 1.
- Assumption 2.
- Assumption 3.

III. Proposed Method

This section details the proposed dynamic timing adjustment algorithm. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose a Dual-Loop Timing Control Architecture.

A. Dual-Loop Architecture

The timing control of this system adopts a “policy-execution separation” dual-loop architecture, ensuring that the timing advancement on the VNF side can flexibly cope with network jitter without inducing unexpected oscillations:

- 1) **Convergence Optimization Loop (Policy):** Dynamically analyzes the statistical data reported by the PNF through the convergence optimization algorithm to determine the global target slot-ahead s_{ahead} .
- 2) **Timing Actuator Loop (Execution):** Executed by an independent VNF timing thread. This thread is the only unit in the system capable of altering the absolute sleep time and slot propagation, translating the upper-layer policy target and low-layer microsecond phase deviations into actual timing advancement.

B. VNF Timing Actuator

The core function of the VNF Timing Actuator is isolation and precise phase execution. Initially, we define the basic mathematical parameters used in the system. Let μ denote the subcarrier spacing parameter (Numerology), and the corresponding slot duration (in microseconds) is defined as T_{slot} :

$$T_{\text{slot}} = \frac{1000}{2^\mu} \quad (1)$$

Let s_{ahead} be the target slot-ahead value, and T_{adv} be the accumulated total phase advance in microseconds. For microsecond-level phase correction, let Δ_{slot} be the integer slot correction provided by the lower layer, and Δ_{pending} be the sub-slot phase deviation (in μs). The algorithm calculates the next absolute wake-up time T_{next} :

$$T_{\text{next}}^{(k)} = T_{\text{next}}^{(k-1)} + T_{\text{slot}} + \Delta_{\text{pending}} \quad (2)$$

Before receiving the initial timing anchor from the PNF, the thread blocks execution via a condition variable. Once active, the thread operates within a mutex-locked environment and triggers sleep according to the absolute monotonic clock calculated in (2), mitigating long-term error accumulation. The condensed control logic is presented in Algorithm 1.

Algorithm 1 VNF Timing Actuator Thread

```

1: procedure TimingActuator( $s_{\text{ahead}}, \mu$ )
2:   Wait for timing_info condition variable
3:    $T_{\text{next}} \leftarrow \text{Now}()$ 
4:    $ID_{\text{DEC}} \leftarrow \text{DEC}(\mu, sfn, slot)$ 
5:   while running do
6:     Lock Mutex
7:      $T_{\text{adv}} \leftarrow s_{\text{ahead}} \cdot T_{\text{slot}}$ 
8:     if  $\Delta_{\text{slot}} \neq 0$  then ▷ Integer correction
9:        $ID_{\text{DEC}} += \Delta_{\text{slot}}$ 
10:      if sync_locked then
11:         $T_{\text{adv}} += \Delta_{\text{slot}}$ 
12:      end if
13:       $\Delta_{\text{slot}} \leftarrow 0$ 
14:    end if
15:     $\Delta_{\text{current}} \leftarrow \Delta_{\text{pending}}$  ▷ Sub-slot correction
16:    if sync_locked then
17:       $T_{\text{adv}} -= \Delta_{\text{current}}$ 
18:    end if
19:     $\Delta_{\text{pending}} \leftarrow 0$ 
20:    Unlock Mutex
21:     $T_{\text{next}} += T_{\text{slot}} + \Delta_{\text{current}}$ 
22:    SleepUntil( $T_{\text{next}}$ )
23:    HandlePeriodicSync
24:     $target \leftarrow \text{wrap}(ID_{\text{DEC}} + s_{\text{ahead}})$ 
25:    while  $last\_mac \neq target$  do ▷ Indication
      catch-up
26:       $last\_mac += 1$  (wrapped)
27:      DeliverToMAC( $last\_mac$ )
28:    end while
29:     $ID_{\text{DEC}} += 1$  (wrapped)
30:  end while
31: end procedure

```

C. Convergence Optimization Policy

To counteract fluctuating network latency and maintain scheduling robustness, this paper proposes a dynamic convergence optimization policy. Upon receiving the timing statistics reported by the PNF, the system incorporates an Exponentially Weighted Moving Average (EWMA) model to estimate the mean latency $\bar{L}$ and jitter variance V_{jitter} . Given the worst-case late arrival L_{worst} in the current observation period, the iterative update formulas are:

$$\bar{L}^{(k)} = \bar{L}^{(k-1)} + \frac{L_{\text{worst}} - \bar{L}^{(k-1)}}{8} \quad (3)$$

$$V_{\text{jitter}}^{(k)} = V_{\text{jitter}}^{(k-1)} + \frac{|L_{\text{worst}} - \bar{L}^{(k-1)}| - V_{\text{jitter}}^{(k-1)}}{4} \quad (4)$$

Before making tracking adjustments, a dynamic safety margin M_{safe} is established to cover the networking jitter, and an absolute boundary B_{abs} is enforced to limit overly aggressive delays:

$$M_{\text{safe}} = \max(1.5 \cdot T_{\text{slot}}, 1.5 \cdot V_{\text{jitter}}) \quad (5)$$

$$B_{\text{abs}} = 2 \cdot T_{\text{slot}} \quad (6)$$

Furthermore, the algorithm designs multiple Panic Criteria triggers (including deadline edge *panic*_{edge}, severe jitter *panic*_{jitter}, statistical anomaly *anomaly*_{stat}, and strict violation *violation*_{strict}). To prevent unexpected oscillation during target transitions, a Directionality Guard is introduced. Additionally, a Penalty Counter & Anti-bounce mechanism filters out pseudo-stable low-latency states to prevent prematurely lowering s_{ahead} . When panic events are detected, the algorithm initiates a fast-attack mode, radically increasing S_{target} to ensure connection continuity. Conversely, when redundant phase advance exists continuously without penalties, a slow-decay backoff is triggered. The overall dynamic adjustment is condensed in Algorithm 3.

Algorithm 2 Delay Manager: Dynamic Scheduler Trigger Timing

Require:

- t_{late} : Worst late timing from PHY (Input)
- T_{slot} : Slot duration in microseconds
- S_{curr} : Current slots ahead (Environment state)

Ensure:

- S_{next} : Adjusted slots ahead (Output: Higher = Earlier, Lower = Later)

- 1: Update EWMA Mean Late (E) and Jitter Variance (V) using t_{late}
 - 2: $B_{safe} \leftarrow (S_{curr} - 1) \times T_{slot} \triangleright$ Absolute safe boundary
 - 3: $C_{panic} \leftarrow (t_{late} \geq T_{slot}) \vee (V > \text{SafeMargin}) \vee$ (Statistical Anomaly)
 - 4: if C_{panic} then
 - 5: $S_{next} \leftarrow S_{curr} + \text{StepUp} \quad \triangleright$ Earlier: Fast attack due to violation/jitter
 - 6: Reset safety counters and apply reduction penalty
 - 7: else if $t_{late} < -B_{safe}$ and Reduction is not locked then
 - 8: $Count_{safe} \leftarrow Count_{safe} + 1$
 - 9: if $Count_{safe} > \text{Decay Threshold}$ then
 - 10: $S_{next} \leftarrow S_{curr} - 1 \quad \triangleright$ Later: Proactive safe latency reduction
 - 11: $Count_{safe} \leftarrow 0$
 - 12: end if
 - 13: else
 - 14: $S_{next} \leftarrow S_{curr} \quad \triangleright$ Maintain: Timing is stable within bounds
 - 15: end if
 - 16: $S_{next} \leftarrow \max(1, \min(S_{next}, 8)) \quad \triangleright$ Apply absolute system bounds
 - 17: return S_{next}
-

Algorithm 3 Dynamic Convergence Optimization Policy

- 1: procedure ConvergenceOpt(L_{worst}, T_{slot})
- 2: $diff \leftarrow L_{worst} - \bar{L}$
- 3: $\bar{L} \leftarrow \bar{L} + diff/8$
- 4: $V_{jitter} \leftarrow V_{jitter} + (|diff| - V_{jitter})/4$
- 5: $B_{abs} \leftarrow 2 \cdot T_{slot}, S_{target} \leftarrow s_{ahead}$
- 6: Evaluate logic flags: $panic, anomaly_{stat}$
- 7: if $L_{worst} < 0$ then
- 8: $anomaly_{stat} \leftarrow false$
- 9: end if
- 10: $\Delta T_{adv} \leftarrow T_{adv} - last_T_{adv}$
- 11: $dir_helpful \leftarrow \text{SignMatch}(\Delta T_{adv}, L_{worst})$
- 12: $S_{prop} \leftarrow \text{clamp}(\text{round}(T_{adv}/T_{slot}), 1, S_{max})$
- 13: if $L_{worst} \geq T_{slot}/2$ then
- 14: $S_{prop} += 1$
- 15: end if
- 16: if $S_{prop} \neq S_{target}$ then
- 17: if $|S_{prop} - S_{target}| > 1$ and not $dir_helpful$ then
- 18: $S_{target} += \text{sign}(S_{prop} - S_{target})$
- 19: else
- 20: $S_{target} \leftarrow S_{prop}$
- 21: end if
- 22: end if
- 23: if $panic$ or $anomaly_{stat}$ then $\triangleright$ Fast-Attack
- 24: $step \leftarrow (violation_{strict})? \lfloor L_{worst}/T_{slot} \rfloor + 2 : 2$
- 25: if $violation_{strict}$ then
- 26: $penalty \leftarrow 10000$

TABLE II
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE III
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

IV. Experimental Results

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduler algorithm, we systematically designed multiple experimental scenarios. Section IV-A outlines the general software and hardware environment used across all tests. Subsequent sections present specific scenarios: Scenario I (IV-B) isolates and verifies the problems caused by non-deterministic latency under standard OpenAirInterface implementations, while Scenario II (IV-C) and subsequent experiments evaluate the robustness of our proposed time-advance control under varying levels of congestion.

A. Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

B. Scenario I: Analysis of Non-Deterministic Latency

Deploying the VNF and PNF on different machines introduces significant challenges. Because the MAC scheduler is separated from the PHY layer, strict slot synchronization is required between these distributed components. However, even when deployed in the exact same network environment, different traffic loads directly impact the latency. Figure 2 illustrates the latency between the VNF scheduler and the PNF under varying traffic loads. As traffic increases, three factors heighten the overall delay: the scheduler requires more processing time, the data packing time lengthens, and the VNF-to-PNF One-Way Delay (OWD) is affected. Consequently, the VNF cannot reliably transmit data exactly at the slot boundary. Instead, it must dynamically advance its scheduling point to compensate for these varying delays and ensure packets arrive at the PNF on time.

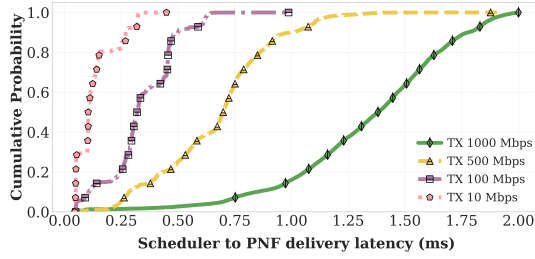


Fig. 2. Observation of latency between VNF scheduler and PNF under different traffic loads.

C. Scenario II: [Placeholder for Future Experiment]

References

- [1] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2833, 2023.
- [2] B. Khan, N. Nidhi, R. R. Paulo, A. Mihovska, and F. J. Velez, "Cost revenue trade-off for the 5g nr small cell network in the sub-6 ghz operating band," in *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*, 2022, pp. 64–69.
- [3] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits proposed for 5g mobile crosshaul networks," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 146–172, 2019.
- [4] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2832, 2023.
- [5] K. Alam, M. A. Habibi et al., "A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 28, 2026.
- [6] S. C. Forum, "5g network fapi (nfapi) specification," *Small Cell Forum*, Tech. Rep. SCF225, 2020.
- [7] —, "5g fapi: Phy api specification," *Small Cell Forum*, Tech. Rep. SCF222, 2021.
- [8] F. Mehran and R. MacKenzie, "Experimental evaluation of multi-vendor virtualized ran using non-ideal fronthaul," in *2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*. IEEE, 2020.
- [9] B. N. Khalaf, W. H. Ali, R. S. Alhumaima, and H. A. J. Alshamary, "Delay aware resource allocation in oran through network optimization," *IET Wireless Sensor Systems*, 2024.
- [10] M. Huang and X. Zhang, "Distributed mac scheduling scheme for c-ran with non-ideal fronthaul in 5g networks," in *Communication Infrastructure Research Lab (Intel Labs China)*. IEEE, 2026.
- [11] B. Akgun et al., "Interference-aware intelligent scheduling for virtualized private 5g networks," *IEEE Access*, vol. 12, 2024.